

Compile-Fused Alignment (CFA)

Compiler-Native Preference Optimization Through Fused Reference-Policy Computation, Compiled Rollouts, and Reward-Integrated Training in NeuralScript

Authors: Brandon Wiemer
Date: April 2026
Status: Research Proposal

Abstract

We propose CFA (**Compile-Fused Alignment**), a compiler-native framework for LLM post-training alignment that treats DPO, GRPO, and RLHF as **compilation targets** rather than runtime orchestration problems. The key bottleneck in alignment training is that frameworks like OpenRLHF/TRL spend 80% of wall-clock time on generation (rollouts) and must hold 2-4 full models in memory simultaneously (policy, reference, reward, critic). CFA exploits NSL's unique position — a compiled language with a built-in compiler-as-reward-model, source AD for arbitrary loss functions, CFIE for fast inference, and WRGA/FASE for memory-efficient training — to eliminate these bottlenecks. The reference model shares weight memory with the policy via copy-on-write compiled bindings, rollout generation uses CFIE's persistent decode kernels (8× faster than vLLM), and the DPO/GRPO loss is fused into a single kernel that never materializes full logit tensors for either model.

1. The Alignment Memory and Speed Crisis

1.1 Model Count Problem

Method	Models in Memory	Memory (7B each, BF16)
SFT	1 (policy)	14 GB + training state
DPO	2 (policy + frozen reference)	28 GB + training state
PPO-RLHF	4 (policy + reference + reward + critic)	56 GB + training state
GRPO	2 (policy + frozen reference)	28 GB + training state
DAPO	2 (policy + frozen reference)	28 GB + training state

For a 7B model with DPO: 28 GB just for weights, plus ~20 GB for training state = ~**48 GB**. A single A100-80GB barely fits this. PPO-RLHF at 56 GB for weights alone is impossible on a single GPU without sharding.

1.2 Generation Bottleneck

GRPO generates $K=8-16$ candidate responses per prompt per training step. OpenRLHF reports that generation accounts for **80% of total training time**. This generation runs the full model autoregressively — identical to inference serving — but current alignment frameworks use unoptimized generation paths (HuggingFace `model.generate()`, not production inference engines).

1.3 The Log-Probability Tax

Both DPO and GRPO require computing per-token log-probabilities under BOTH the policy and reference models for the same sequences. In PyTorch:

```
python

# DPO: two full forward passes for the same input
policy_logits = policy_model(chosen_input)           # forward pass 1
ref_logits = ref_model(chosen_input)                 # forward pass 2
policy_logits_rej = policy_model(rejected_input)      # forward pass 3
ref_logits_rej = ref_model(rejected_input)            # forward pass 4

# 4 full forward passes per DPO step!
```

Each forward pass materializes a `[batch, seq, vocab]` logits tensor, computes `log_softmax`, then indexes into the label position. The full logit tensor (192 KB per token at `vocab=49152`) is created and immediately discarded after extracting a single scalar.

2. Six Compiler-Native Alignment Innovations

2.1 Innovation 1: Copy-on-Write Reference Model

The problem: The reference model is a frozen copy of the SFT checkpoint. In PyTorch, this means loading two complete copies of all model weights — doubling memory.

CFA's approach: The compiler generates a copy-on-write (CoW) binding between policy and reference models. At the start of alignment training, both share the SAME weight tensors. As the policy updates its weights during training, only the modified layers diverge:

Initial state (after SFT):

```
policy.layer[0].Wq → [shared tensor A] ← ref.layer[0].Wq
policy.layer[0].Wk → [shared tensor B] ← ref.layer[0].Wk
...
```

After 100 training steps:

```
policy.layer[0].Wq → [updated tensor A']  ref.layer[0].Wq → [original
tensor A]
policy.layer[0].Wk → [updated tensor B']  ref.layer[0].Wk → [original
tensor B]
policy.layer[5].Wq → [shared tensor F] ← ref.layer[5].Wq (not yet
diverged)
...
```

Memory savings: In early training, policy weights diverge slowly from the reference. After N steps, typically only 10-30% of layers have diverged significantly. The CoW scheme saves 70-90% of the reference model's memory cost in early training.

The compiler implements this by maintaining a per-layer divergence flag. When the optimizer updates a parameter, the compiled optimizer step checks the flag and, if the parameter is still shared, allocates a new buffer and copies the original (the "copy" in CoW) before applying the update. This check is a single branch instruction — negligible cost.

Late training: Eventually all layers diverge and memory equals the naive approach. But the savings during the critical early phase (when memory is tightest because the model hasn't settled yet) are significant.

For PEFT/LoRA alignment (common in practice): the base weights NEVER diverge — only adapter weights change. CoW means the reference model adds **zero additional memory** for base weights.

2.2 Innovation 2: Fused Reference-Policy Log-Probability Kernel

The problem: Computing $\log \pi_{\theta}(y|x)$ requires a full forward pass that produces a [batch, seq, vocab] logits tensor, followed by $\log_softmax$, followed by gathering the log-prob at each label position. The full logit tensor is wasted — only one value per position is used.

CFA's approach: Extend the fused decode-sample kernel from CFIE to a **fused log-prob kernel** that computes per-token log-probabilities without materializing the full logits tensor:

Fused Log-Prob Kernel (per layer's LM head):

Input: hidden [B, S, d_model], W_lm_head [d_model, vocab], labels [B, S]

Output: log_probs [B, S] (one scalar per token – not per vocab!)

```
for vocab_tile in 0..ceil(vocab / TILE):
    logits_tile = hidden @ W_tile           # [B, S, TILE] in SMEM

    # Online log-sum-exp accumulation (numerically stable)
    for each (b, s):
        tile_max = max(logits_tile[b, s, :])
        if tile_max > running_max[b, s]:
            running_sum_exp[b, s] *= exp(running_max[b, s] - tile_max)
            running_max[b, s] = tile_max
        running_sum_exp[b, s] += sum(exp(logits_tile[b, s, :] - running_max[b, s]))

    # If label falls in this tile, save its logit
    if labels[b, s] - tile_offset is in [0, TILE):
        label_logit[b, s] = logits_tile[b, s, labels[b, s] - tile_offset]

# Final: log_prob = label_logit - running_max - log(running_sum_exp)
log_probs[b, s] = label_logit[b, s] - running_max[b, s] - log(running_sum_exp[b, s])
```

HBM savings: The [batch, seq, vocab] logits tensor (potentially 1+ GB for batch=4, seq=2048, vocab=49152) is NEVER written to HBM. Only [batch, seq] log-probs (32 KB) are output.
~30,000× reduction in output traffic.

This kernel runs for BOTH the policy and reference models. For the reference model (frozen, no grad), no activations are saved — the kernel runs forward-only and discards everything except the log-probs.

2.3 Innovation 3: Fused DPO/GRPO Loss Kernel

DPO loss:

$$L_{DPO} = -\log \sigma(\beta \times (\log \pi_{\theta}(y_w|x) - \log \pi_{ref}(y_w|x) - \log \pi_{\theta}(y_l|x) + \log \pi_{ref}(y_l|x)))$$

CFA fuses this entire computation:

Fused DPO Loss Kernel:

Inputs:

```
policy_hidden_chosen, policy_hidden_rejected  [B, S, d]
ref_hidden_chosen, ref_hidden_rejected        [B, S, d]
W_lm_head (shared between policy and ref if CoW not diverged)
labels_chosen, labels_rejected                [B, S]
 $\beta$  (compile-time constant)
```

Output: scalar loss, gradient w.r.t. policy_hidden_chosen and policy_hidden_rejected

```
# Compute all 4 log-prob sets in one kernel:
# (tilted matmul with online log-sum-exp, as in Innovation 2)
policy_logprob_chosen = fused_logprob(policy_hidden_chosen, W_lm_head,
labels_chosen)
ref_logprob_chosen = fused_logprob(ref_hidden_chosen, W_lm_head, labels_chosen)
policy_logprob_rejected = fused_logprob(policy_hidden_rejected, W_lm_head,
labels_rejected)
ref_logprob_rejected = fused_logprob(ref_hidden_rejected, W_lm_head,
labels_rejected)

# DPO loss (in registers)
reward_diff =  $\beta \times ((\text{policy\_logprob\_chosen} - \text{ref\_logprob\_chosen})$ 
                $- (\text{policy\_logprob\_rejected} - \text{ref\_logprob\_rejected}))$ 
loss = -log_sigmoid(reward_diff)

# Gradient w.r.t. policy log-probs (analytical, in registers)
d_loss =  $\beta \times \text{sigmoid}(-\text{reward\_diff})$ 
# Backprop through the log-prob computation to get grad w.r.t. hidden
grad_policy_hidden_chosen = d_loss  $\times$  d_logprob_d_hidden(...)
grad_policy_hidden_rejected = -d_loss  $\times$  d_logprob_d_hidden(...)
```

Memory savings vs PyTorch TRL DPO:

Component	PyTorch TRL	CFA Fused
Policy logits chosen [B,S,V]	384 MB	0 (never materialized)
Policy logits rejected [B,S,V]	384 MB	0
Ref logits chosen [B,S,V]	384 MB	0
Ref logits rejected [B,S,V]	384 MB	0
Total logit memory	1,536 MB	0

2.4 Innovation 4: CFIE-Accelerated Rollout Generation

The problem: GRPO generates K=8-16 responses per prompt. This is 80% of training time. Current frameworks use `model.generate()` which launches ~500 kernels per token per layer.

CFA's approach: Rollout generation uses CFIE's compiled inference engine:

- **Persistent decode blocks:** 1 kernel launch per layer, not 30
- **Fused decode-sample:** LM head + softmax + sampling in one kernel
- **Static KV layout:** No PagedAttention block table lookups during generation
- **Compiled speculative decoding:** Draft model baked into the binary

For GRPO with K=8 rollouts, each generating ~512 tokens:

Phase	PyTorch generate()	CFA + CFIE
Kernel launches per token	~3,780 (32-layer model)	~34
Sampling overhead per token	~30 μ s	~2 μ s (fused)
Generation time per rollout	~16 seconds	~2 seconds
Total K=8 generation	~128 seconds	~16 seconds
With speculative (K=5, tree)	—	~5 seconds

8-25× faster rollout generation. Since rollouts are 80% of training time, this translates to **3-5× faster total GRPO training.**

2.5 Innovation 5: Compiler-as-Reward-Model (NSL Unique)

NSL's coder-rl design already uses the **NSL compiler pipeline as the reward function:**

```
Stage 1: Parse           → reward = 0.2
Stage 2: Semantic check  → reward = 0.4
Stage 3: Shape check     → reward = 0.6
Stage 4: Full compile    → reward = 0.8
Stage 5: Execute + test  → reward = 1.0
```

CFA compiles the reward function into the training binary. The compiler pipeline runs in the SAME Rust process as the training loop — no subprocess spawning, no IPC overhead:

```

nsl

align(model=m, method=grpo):
    reward:
        mode = compiler                # use NSL compiler as reward
        stages = [parse, semantic, shape, compile, run]
        partial_credit = true
        curriculum = true              # start with easy tasks, progress to hard
    ...

```

The compiler generates a `reward_fn` that calls `nsl_parse()`, `nsl_semantic_check()`, `nsl_shape_check()`, and optionally `nsl_codegen()` and `nsl_execute()` — all as function calls within the compiled binary, not as subprocess invocations.

Why this is uniquely NSL: No other ML framework has a compiler that can serve as a differentiable-adjacent reward function. The compiler provides a rich, hierarchical, deterministic reward signal with zero stochasticity — exactly what RLVR needs.

2.6 Innovation 6: Source-AD for Arbitrary Alignment Losses

New alignment algorithms emerge frequently (DPO → SimPO → KTO → ORPO → DAPO → GSPO). In PyTorch, each requires a custom training loop. In NSL, source AD (M40) automatically generates backward passes for ANY loss function expressed in NSL:

ns!

```
# SimPO loss – no reference model needed!
fn simpo_loss(policy_logprobs_chosen: Tensor, policy_logprobs_rejected: Tensor,
              chosen_len: int, rejected_len: int,  $\beta$ : f32,  $\gamma$ : f32) -> Tensor:
    let avg_chosen = policy_logprobs_chosen.sum() / chosen_len
    let avg_rejected = policy_logprobs_rejected.sum() / rejected_len
    return -log_sigmoid( $\beta$  * (avg_chosen - avg_rejected) -  $\gamma$ )

# KTO loss – binary feedback, no preference pairs!
fn kto_loss(policy_logprob: Tensor, ref_logprob: Tensor,
            is_preferred: bool,  $\beta$ : f32) -> Tensor:
    let kl = policy_logprob - ref_logprob
    if is_preferred:
        return -log_sigmoid( $\beta$  * kl)
    else:
        return -log_sigmoid(- $\beta$  * kl)

# DAPO token-level policy gradient – critical for long CoT
fn dapooloss(policy_logprobs: Tensor, advantages: Tensor,
             old_logprobs: Tensor, clip_low: f32, clip_high: f32) -> Tensor:
    let ratio = exp(policy_logprobs - old_logprobs)
    let clipped = clamp(ratio, clip_low, clip_high)
    let pg_loss = -min(ratio * advantages, clipped * advantages)
    return pg_loss.mean() # token-level, not sequence-level
```

Source AD generates backward passes for all of these automatically. The user doesn't need to derive gradients for new alignment algorithms — they just write the loss function in NSL and the compiler handles the rest.

3. NSL Language Surface

nsi

— DPO Alignment —

```
let policy = NSLCoder.load("sft_checkpoint.nslm")
let reference = policy.frozen_copy() # CoW - shares memory until policy diverges

align(model=policy, reference=reference, method=dpo, epochs=3):
  optimizer: AdamW(lr=5e-7, weight_decay=0.01)
  loss:
    beta = 0.1
    label_smoothing = 0.0
    loss_type = sigmoid          # or hinge, ipo

  data:
    source = preference_dataset # (prompt, chosen, rejected) triples
    batch_size = 4
    packing = true              # PCA packing for preference pairs

  step(batch):
    let loss = dpo_step(policy, reference, batch)
```

— GRPO Alignment with Compiler Reward —

```
align(model=policy, method=grpo, epochs=1):
  optimizer: AdamW(lr=1e-6)
  rollout:
    k = 8                      # generate 8 candidates per prompt
    max_tokens = 512
    temperature = 0.7
    speculative = true          # use CFIE speculative decoding
  reward:
    mode = compiler             # NSL compiler as reward
    curriculum = true
  loss:
    beta = 0.04                 # KL penalty
    clip = [0.8, 1.2]          # DAPO-style clip range
    token_level = true          # DAPO token-level PG

  step(prompt):
    # Compiled pipeline:
    # 1. Generate K=8 rollouts using CFIE (persistent kernels, fused sampling)
    # 2. Score through compiler pipeline (in-process, no subprocess)
    # 3. Compute GRPO advantages (group-relative normalization)
    # 4. Compute token-level policy gradient loss
    # 5. Backward with FASE (fused optimizer, per-layer)
    let loss = grpo_step(policy, prompt)
```

bash

```
$ nsl build --align=dpo --weights sft.safetensors --target 8xH100 align.nsl
```

=== CFA Alignment Build Report ===

Method: DPO

Policy: NSLCoder-50M (48.8M params)

Reference: CoW (shared memory until divergence)

Optimizations:

- [1] CoW reference: 0 additional memory at start (saves 97.6 MB)
- [2] Fused log-prob kernel: logits never materialized (saves 1.5 GB)
- [3] Fused DPO loss: 4 log-prob computations + loss + grad in 1 kernel
- [4] WRGA dead grad: reference backward absent from binary
- [5] FASE: policy optimizer fused per-layer
- [6] PCA: preference pairs packed with segment-ID masking

Memory per GPU:

Policy weights (BF16):	97.6 MB
Reference weights (CoW):	0 MB (shared) → 97.6 MB (fully diverged)
Policy optimizer (CPDT mixed):	37 MB
Policy activations (CSHA):	128 MB
Reference activations:	0 (forward-only, no save)
Logit tensors:	0 (fused log-prob)
CUDA context:	800 MB

TOTAL:	~1.1 GB (start) → ~1.2 GB (diverged)
--------	--------------------------------------

Standard PyTorch TRL DPO:	~3.8 GB
---------------------------	---------

Memory reduction:	3.2-3.5× smaller
-------------------	------------------

4. Integration with Previous Papers

Paper	CFA Usage
WRGA	LoRA-DPO: only adapter weights diverge from reference → CoW base weights = zero overhead
CSHA	Fused attention in both policy and reference forward passes
FASE	Policy optimizer fused per-layer during backward
PCA	Preference pairs (chosen + rejected) packed with segment-ID masking
CEP	Pruned policy+reference share pruned architecture (same binary)
CPDT	Multi-GPU DPO: sharded policy + sharded reference with CoW awareness
WGGO	Alignment-specific decisions in ILP: CoW threshold, rollout batch size, loss fusion level
CFIE	GRPO rollout generation uses persistent decode kernels + speculative decoding
CIDP	Online quality scoring applied to preference data

5. Why PyTorch/TRL/OpenRLHF Cannot Do This

Capability	PyTorch (TRL/OpenRLHF)	CFA
Reference model memory	Full copy (2× model size)	CoW (0× initially, diverges gradually)
Log-prob computation	Full [B,S,V] logits → log_softmax → gather	Fused kernel: 1 scalar per token, logits never in HBM
DPO loss	4 separate forward passes + Python loss	1 fused kernel: 4 log-probs + loss + gradient
GRPO rollout speed	HuggingFace generate() or vLLM	CFIE persistent kernels + speculative (8-25× faster)
Reward model	External process / API call	Compiler pipeline as in-process function call
New alignment algorithms	Custom training loop per algorithm	Source AD: write loss function → backward auto-generated
Reference backward	Saved activations (wasted memory)	Physically absent from binary (WRGA dead grad)

6. Implementation Plan

Component	Location	LOC (est.)	Description
CoW reference manager	<code>ns1-codegen/src/cfa_cow.rs</code>	400	Copy-on-write weight sharing with divergence tracking
Fused log-prob kernel	<code>ns1-codegen/src/cfa_logprob.rs</code>	500	Tiled matmul + online log-sum-exp
Fused DPO/GRPO loss	<code>ns1-codegen/src/cfa_loss.rs</code>	600	Combined log-prob + loss + gradient kernel
GRPO rollout pipeline	<code>ns1-codegen/src/cfa_rollout.rs</code>	500	CFIE-accelerated generation for K candidates
Compiler reward	<code>ns1-codegen/src/cfa_reward.rs</code>	300	In-process compiler pipeline as reward function
Align block codegen	<code>ns1-codegen/src/align.rs</code>	500	Parse and compile <code>align()</code> block
SimPO/KTO/DAPO losses	<code>ns1-stdlib/src/align/</code>	200	Standard alignment loss functions for source AD
WGGO integration	<code>ns1-codegen/src/wggo.rs</code>	150	Alignment variables in ILP

Total: ~3,150 LOC. Dependencies: `source_ad.rs`, `fused_linear_ce.rs`, `cfie_fused_sample.rs`, `serve.rs`, `fase.rs`, `cost_model.rs`

7. Conclusion

CFA completes the eleventh and final paper in the NSL compiler-native optimization suite:

#	Paper	Domain
1	WRGA	PEFT / Adapters
2	CSHA	Attention Fusion
3	CFTP	Gradient Accumulation + Packing
4	CEP	Pruning + NAS
5	CPDT	Distributed Training
6	WGGO	Global ILP Solver
7	CFIE	Inference Optimization
8	CPKD	Knowledge Distillation
9	CIDP	Data Quality + Filtering
10	CFA	RLHF / DPO / GRPO Alignment

CFA's central insight: **alignment is a joint inference-and-training problem that a compiler can fuse**. The rollout phase IS inference (CFIE applies). The training phase uses a reference model that shares memory with the policy (CoW). The loss function involves log-probabilities that never need full logit tensors (fused kernel). And NSL's own compiler serves as a reward function for code generation tasks. No existing alignment framework can exploit all four of these observations simultaneously because they span the inference/training/compilation boundary.

References

1. Rafailov, R., et al. "Direct Preference Optimization: Your Language Model Is Secretly a Reward Model." NeurIPS 2023.
2. Shao, Z., et al. "DeepSeekMath: Pushing the Limits of Mathematical Reasoning (GRPO)." 2024.
3. Yu, Q., et al. "DAPO: An Open-Source LLM Reinforcement Learning System." 2025.
4. Meng, Y., et al. "SimPO: Simple Preference Optimization with a Reference-Free Reward." NeurIPS 2024.
5. Ethayarajh, K., et al. "KTO: Model Alignment as Prospect Theoretic Optimization." ICML 2024.
6. OpenRLHF Team. "OpenRLHF: An Easy-to-use, Scalable and High-performance RLHF Framework." 2024.

